

LECTURE 23

Views and Materialized Views

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

What We'll Cover

- What a view is: a virtual table with no stored data of its own
- Creating views with `CREATE VIEW ... AS SELECT ...`
- Querying a view exactly like a base table
- Which views are updatable, and which cannot be
- Advantages and disadvantages of ordinary views
- Materialized views: what “materialized” actually changes
- Creating, refreshing, and choosing materialized views

Views: a Virtual Table

- A view's definition is a stored query — it has no rows of its own on disk.
- Every query against a view re-runs the underlying query against current base-table data.
- Nothing is duplicated: the view is a lens onto the base tables, not a copy of them.

Think of a view as a saved **SELECT** with a name

Any application that queries the view gets the current answer, computed from the live base table. Nothing is copied, so nothing can go stale.

The Running Example: the Staff Relation

A branch manager should only see staff at their own branch, not the whole company.

Staff (staffNo, name, position, salary, branchNo)

staffNo	name	position	salary	branchNo
SL21	John White	Manager	30000	B005
SG37	Ann Beech	Assistant	12000	B003
SG14	David Ford	Supervisor	18000	B003
SA9	Mary Howe	Assistant	9000	B007

SG37 and SG14 both belong to branch B003 — that is the set a B003 manager should see.

Creating a View: Filtering Rows

Filter rows for one branch. Nothing is copied; the name is bound to the SELECT.

Filter rows: one branch only

```
CREATE VIEW Branch003Staff AS
  SELECT staffNo, name, position, salary
  FROM Staff
  WHERE branchNo = 'B003';
```

Each reference re-runs the query

Every time Branch003Staff is referenced, the DBMS evaluates this SELECT against the live Staff table.

Creating a View: Joining Tables

A view can join base tables, so applications see one simple shape.

Join two base tables

```
CREATE VIEW StaffWithBranchCity AS
  SELECT s.staffNo, s.name, s.position, b.city
  FROM Staff s JOIN Branch b ON s.branchNo = b.branchNo;
```

Still no copied data

The join runs on every query. The view only names the join; the rows come from Staff and Branch.

Creating a View: Aggregating

A view can also aggregate: one row per branch, computed from Staff.

Aggregate with GROUP BY

```
CREATE VIEW BranchSalaryTotals AS
  SELECT branchNo, COUNT(*) AS staffCount,
         SUM(salary) AS totalSalary
  FROM Staff GROUP BY branchNo;
```

Row-level detail is summarized away

Each row of this view is a group. The individual Staff rows are no longer visible through it.

Querying Views

A view is queried exactly like a base table. Application code does not need to know it is a view.

Query the view like a table

```
SELECT * FROM Branch003Staff  
WHERE salary > 10000  
ORDER BY salary DESC;
```

- You can join a view with a base table, or with another view.
- The DBMS re-evaluates both underlying queries and combines the results each time.

When Can You Update Through a View?

A view is updatable only if every change can be translated unambiguously onto one base table:

- Built from a single base table (no join)
- No DISTINCT
- No GROUP BY, HAVING, or aggregate function (SUM, COUNT, AVG, ...)
- No set operations (UNION, INTERSECT, EXCEPT)
- Includes every column that has no default and disallows NULL in the base table

An Updatable View in Practice

Branch003Staff qualifies: a single-table SELECT with no aggregates.

Legal update through the view

```
UPDATE Branch003Staff  
SET salary = salary * 1.05  
WHERE staffNo = 'SG37';
```

Behaves exactly as if you had updated Staff directly

The DBMS maps the change back onto the one row of Staff with staffNo SG37.

Views That Cannot Be Updated

BranchSalaryTotals is not updatable. There is no row-level value to change.

Rejected by the DBMS

```
UPDATE BranchSalaryTotals SET totalSalary = 50000
WHERE branchNo = 'B003';
-- ERROR: cannot update a view that contains aggregate functions
```

- A join view such as StaffWithBranchCity is also non-updatable in most systems: which base table would the change apply to?
- Some systems offer INSTEAD OF triggers, but that custom logic is application-defined, not automatic.

Advantages of Views

Ordinary views buy three things, at no storage cost:

Security

Grant access to a view's rows or columns without exposing the whole base table (Lecture 24).

Simplicity

Hide complex joins and aggregations behind one simple name.

Logical data independence

Restructure the base schema, update the view definition, and applications keep working.

Disadvantages of Views

The price of a view is recomputation, and restricted updatability:

Performance overhead

The underlying query re-runs on every reference. An expensive join or aggregate is paid again each time.

Update restrictions

Many useful views (joins, aggregates, DISTINCT) are not updatable, forcing writes back to base tables.

Views save you storage, not computation

The next topic, materialized views, trades storage for speed.

Ordinary View vs. Materialized View

Plain VIEW

Re-runs the underlying SELECT against the base tables on every query. The result is always fresh.

Cost: recomputation on every access.

MATERIALIZED VIEW

Reads the stored result directly, like a table. Fast, but possibly stale until it is refreshed.

Cost: storage, plus a refresh step to stay current.

Creating a Materialized View

The syntax is nearly identical. Only the keyword changes, but the effect is very different.

Stored result set

```
CREATE MATERIALIZED VIEW BranchSalaryTotals_MV AS
  SELECT branchNo, COUNT(*) AS staffCount,
         SUM(salary) AS totalSalary
  FROM Staff GROUP BY branchNo;
```

- The result now occupies real storage, sized like the query's output.
- It does NOT reflect a later UPDATE to Staff until it is explicitly refreshed.

Refreshing Materialized Views

Because the stored data can drift from the base tables, every materialized view needs a refresh.

Refreshing on demand

```
-- On-demand: blocks until the refresh completes  
REFRESH MATERIALIZED VIEW BranchSalaryTotals_MV;  
  
-- On-demand: readers keep using the old data while it refreshes  
REFRESH MATERIALIZED VIEW CONCURRENTLY BranchSalaryTotals_MV;
```

Scheduled and on-commit refresh are next

A scheduler can call REFRESH on a timer; some systems refresh automatically on each commit.

Choosing a Refresh Strategy

The refresh strategy is the most important design decision for a materialized view.

Strategy	When it runs	Best for
On-demand	Manually, whenever REFRESH is issued	Occasional reports where some staleness is acceptable
Scheduled	On a timer (nightly, hourly) via a job scheduler	Dashboards with a known, bounded staleness window
On-commit	Automatically, as part of every committing transaction	Data that must never be stale, at the cost of slower writes

No free lunch

On-commit refresh removes staleness, but every write to the base table pays for it.

Advantages of Materialized Views

Materialized views move the expensive work to refresh time:

Query performance

Expensive joins and aggregations are computed once at refresh time, not on every read.

Lower load on base tables

Read-heavy reporting and analytics hit the stored result instead of re-deriving it from base tables.

Disadvantages of Materialized Views

The costs are staleness, storage, and refresh work:

Staleness

Data can lag behind the base tables between refreshes. Unacceptable for, e.g., real-time balances.

Storage cost

The result set is physically duplicated.

Refresh overhead

A full, non-incremental refresh can be as expensive as the original query.

Choosing Between a View and a Materialized View

Default to an ordinary view.

Reach for a materialized view only when profiling shows an expensive query is run often enough, relative to how often its data changes, that recomputing it is measurably wasteful — and the application can tolerate the staleness window between refreshes.

If zero staleness is required

REFRESH ON COMMIT narrows the window to zero, at the cost of slower writes to the base tables. There is no free lunch: it is a trade-off you pick deliberately.

Key Takeaways

- A view is a virtual table: a stored, named query with no data of its own, re-evaluated on every reference.
- Views are queried like base tables, but only simple single-table, non-aggregate views are updatable.
- Ordinary views buy security, simplicity, and logical data independence, at the cost of recomputation.
- A materialized view stores its result, trading storage and staleness for query speed.
- The refresh strategy (on-demand, scheduled, or on-commit) decides how stale the stored data may become.